Google Cloud Network Architecture & VPC Service Controls (Shared VPC, Private Access, Perimeters)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The outcome . . . 4

The approach . . . 4

Why it is safe and governed . . . 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

Reference design . . . 6

Benefits . . . 6

Limitations & boundaries . . . 6

Deployment model . . . 7

Requirements 7

1. Network administration roles at the host project and organization . . . 7

2. A dedicated Shared VPC host project (and target service projects) . . . 8

3. Non-overlapping IP address plan (subnets + secondary ranges) . . . 8

4. Private access foundations: DNS and the restricted API VIP route . . . 8

5. Access Context Manager access policy (the container for access levels & perimeters) . . . 9

6. Required APIs enabled on the in-scope projects . . . 9

7. A logging & monitoring sink for VPC-SC and firewall audit . . . 10

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions 10

1. A Google Cloud Organization linked to Cloud Identity / Workspace . . . 10

2. Organization-level administrative access . . . 11

3. A Cloud Billing account . . . 11

4. The gcloud / IaC toolchain and API enablement . . . 11

5. A resource-hierarchy, naming & environment design . . . 12

6. Security, network & governance stakeholders and policy . . . 12

Implementation 12

Stage 0 — Assess (read-only) . . . 13

Stage 1 — Design (no changes) . . . 13

Stage 2 — Configure the Shared VPC + private access (pilot host first) . . . 13

Stage 3 — Firewall & egress policy (deny-by-default) . . . 14

Stage 4 — VPC Service Controls perimeter (DRY-RUN first) . . . 15

Stage 5 — Enforce (production change, with rollback) . . . 15

Stage 6 — Expand . . . 16

Testing & Validation 16

T1 — Shared VPC topology is correct . . . 16
T2 — Private Google Access works from an internal-only VM . . . 16
T3 — Egress is deny-by-default . . . 17
T4 — Perimeter dry-run logs a known exfiltration attempt . . . 17
T5 — Legitimate flows are covered before enforcement (Gate G2) . . . 17
T6 — Enforcement blocks exfiltration and preserves legitimate access . . . 17
T7 — Access level gates by trusted condition . . . 18
T8 — Config helper output matches live state (drift check) . . . 18
Troubleshooting 18
Private access . . . 18
Shared VPC . . . 19
Firewall / egress . . . 19
VPC Service Controls — dry-run . . . 19
VPC Service Controls — enforced . . . 20
Permissions & policy . . . 20
Tooling . . . 20
Best Practices 21
Perimeters . . . 21
Access levels & identity . . . 21
Shared VPC & private access . . . 21
Automation & secrets . . . 22
Operations . . . 22
Deliverables Summary 22
Documentation . . . 22
Python tools (5) — `gcloud` / Cloud Client Library automation . . . 23
Named artifacts (reused across docs, tests, and tools) . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary

The outcome

The organization gets an **enterprise Google Cloud network that is private by default and bounded against data exfiltration**. Workloads reach Google services (Cloud Storage, BigQuery, and the AI platform services) over Google's private backbone rather than the public internet, network egress is **denied by default** and allowed only by named exception, and **VPC Service Controls perimeters** draw a logical boundary around Google-managed services so that data cannot be copied out of the trusted boundary — even by a caller who holds valid credentials. The flat, publicly reachable default network is replaced by a governed one where the exfiltration paths are closed.

The approach

The build layers four controls, each verified before the next: a **Shared VPC** topology that keeps network administration central while application teams deploy independently; **private access** to Google APIs via Private Google Access and Private Service Connect; a **deny-by-default firewall and egress policy**; and **VPC Service Controls perimeters** that stop cross-boundary data movement at the API layer. The whole engagement follows a disciplined, pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates.

Five Python tools (using the `gcloud` CLI and Google Cloud Client Libraries) automate the work: assessing current exposure, provisioning the Shared VPC, validating private access, building the perimeter, and producing the go/no-go validation report.

Why it is safe and governed

The defining safeguard is **dry-run first**. Every perimeter is created in **dry-run (audit) mode**, where it *logs what it would block without blocking anything*. Those logs are reviewed to find and whitelist legitimate business flows, and only when the evidence shows no legitimate flow would break — and a deliberate exfiltration attempt is confirmed as a would-deny — is the perimeter **enforced**. A rehearsed **rollback to dry-run** is the emergency brake if enforcement ever surprises a workload.

Automation is **read-only or dry-run by default**, requires an explicit opt-in to make any change, and contains **no credentials or keys** in code. Org-wide guardrails land under change control with the named security and network approvers. Regulatory determinations (data residency, retention) remain with the organization's security, legal, and compliance functions; this solution delivers the technical method and the evidence. The result: **workloads connect privately, data stays inside enforced perimeters, and every production change is proven before it takes effect.**

Solution Overview

What this is

This solution designs and stands up the **enterprise network foundation** on Google Cloud Platform for a large organization (80k–150k+ users) that is moving from Microsoft 365/Entra to Google Workspace,

adopting Google Cloud, and deploying Gemini Enterprise. It replaces the default, flat, publicly reachable network posture with a **governed, private, perimeter-bounded** network:

• a **Shared VPC** topology (one host project owns the network; service projects attach to it),

• **private connectivity to Google APIs** — Private Google Access and Private Service Connect — so workloads never traverse the public internet to reach Cloud Storage, BigQuery, and other services,

• **hybrid connectivity patterns** (Cloud VPN / Cloud Interconnect / Network Connectivity Center) referenced at the design level for on-prem and multi-cloud reach,

• a **firewall and egress policy** built on hierarchical firewall policies, network firewall policies, and (where used) org-policy egress constraints, and

• **VPC Service Controls (VPC-SC) perimeters** that draw a logical boundary around Google-managed services so that **data cannot be copied out** of the trusted boundary — introduced **in dry-run (audit) mode first**, validated against real traffic, then **enforced**.

The result is a network where workloads connect to Google services **privately**, egress is **explicitly controlled**, and API-level data exfiltration paths are **closed** — safe by design and governed by policy rather than by ad-hoc firewall rules.

How it works

The design layers four controls, each independently valuable and each verified before the next:

1. **Shared VPC topology.** A dedicated **host project** (`Net-Host – Prod – Shared-VPC`) owns the VPC network, subnets, routes, and firewall policies. Workload **service projects** are attached and consume shared subnets. This centralizes network administration with the network team while letting application teams deploy into their own projects — separation of duties without separate networks. Subnets are regional, sized with room to grow, and carry secondary ranges for GKE Pods/Services where container platforms are in scope.

1. **Private access to Google APIs. Private Google Access** is enabled per-subnet so VMs with only internal IPs reach Google APIs via `private.googleapis.com` / `restricted.googleapis.com` virtual IP ranges (the **restricted** VIP is the one that keeps traffic inside a VPC-SC perimeter). **Private Service Connect (PSC)** endpoints provide a private, org-owned IP for the Google APIs bundle, giving DNS you control and a clean path that composes with VPC-SC. Public egress to Google APIs is removed as a dependency.

1. **Firewall & egress policy.** A **hierarchical firewall policy** at the organization/folder level sets non-negotiable guardrails (deny-by-default egress, allow the Google API VIP ranges, deny known-bad ranges), and **network firewall policies** at the VPC level implement workload-specific allows. The default posture is **deny egress**; each allow is named and justified. This is the network-layer complement to the API-layer control below.

1. **VPC Service Controls perimeters.** A **service perimeter** (`Perimeter – Prod – v1`) enumerates the projects inside the boundary and the Google services it protects (for example Cloud Storage, BigQuery, and — verify current coverage for your services — the Vertex AI / Gemini APIs). Requests that

would move protected data **across** the boundary (e.g. copying a bucket to a project outside the perimeter, or reading BigQuery from an unmanaged network) are denied. **Access levels** (Access Context Manager) describe the trusted conditions (corp IP ranges, managed device posture) under which ingress/egress is allowed. The perimeter is created in **dry-run mode** so it *logs* what it *would* block without denying anything, the logs are reviewed to find legitimate flows that need ingress/egress rules, and only then is it **enforced**.

Reference design

```
Organization
 ■■■ Folder: Network
      ■■■ Host project:  Net-Host – Prod – Shared-VPC   (owns VPC, subnets, firewall policy)
      ■     ■■■ VPC: vpc-prod-shared
      ■           ■■■ subnet: sn-prod-use4   (Private Google Access ON; restricted VIP route)
      ■           ■■■ subnet: sn-prod-euw1   (secondary ranges for GKE where in scope)
      ■           ■■■ PSC endpoint: psc-googleapis  → all-apis / vpc-sc bundle
      ■■■ Service projects (attached):  App-A – Prod ,  Data-A – Prod  …

Access Context Manager
 ■■■ Access Level: Access Level – Corp-Managed – v1   (corp IP ranges + device posture)

VPC Service Controls
 ■■■ Service Perimeter: Perimeter – Prod – v1  (DRY-RUN → ENFORCED)
       restricted services:  storage, bigquery, aiplatform, … (verify coverage in your org)
       ingress/egress rules referencing Access Level – Corp-Managed – v1
```

Hybrid reach (Cloud VPN, Dedicated/Partner Interconnect, or Network Connectivity Center hubs) is attached to the host VPC and is designed so on-prem sources that must reach protected services do so **through** an ingress rule tied to the access level — never around the perimeter.

Benefits

- **Exfiltration paths closed at the API layer.** VPC-SC denies data movement across the boundary

even when a caller has valid IAM permissions — mitigating stolen-credential and misconfiguration data-loss scenarios that IAM alone cannot.

- **Private by default.** Workloads reach Google services over Google's backbone via the restricted

VIP / PSC, not the public internet — smaller attack surface and cleaner compliance story.

- **Separation of duties.** Network is owned centrally (host project); apps deploy independently

(service projects) without each team running its own network.

- **Change-safe.** Every perimeter and policy change lands in **dry-run/audit first**, with logs

proving what it would block, so enforcement never silently breaks production.

- **Automatable and auditable.** The included Python tools assess exposure, provision the topology in

dry-run, build perimeters in dry-run, and produce a validation report — read-only or dry-run by default, no secrets in code.

Limitations & boundaries

- **VPC-SC protects Google-managed services, not your own VMs' data plane.** It governs access to the

service APIs in its coverage list; it is not a host firewall or a DLP content scanner. Pair it with firewall policy, IAM least-privilege, and (separately) DLP.

• **Service coverage is version-dependent.** Which Google services VPC-SC supports, and whether a service is "supported/preview/unsupported," changes over time. This solution names representative services; **verify current VPC-SC coverage for your exact services in your organization** before relying on the perimeter for them (Gemini Enterprise / Vertex AI coverage in particular is evolving — confirm in-tenant).

• **IaC is at the pattern level.** Terraform / Cloud Foundation Toolkit blueprints are shown as representative patterns; provider/module internals are deferred to the tool's own documentation.

• **Regulatory determinations are deferred.** Data-residency, retention, and compliance *decisions* belong to the organization's security, legal, and compliance functions; this solution provides the technical method and the evidence.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a documented rollback for every org-wide or production-affecting change (the perimeter enforcement step above all). Nothing is enforced org-wide until it has passed its gate in a named pilot scope and its dry-run logs have been reviewed.

Requirements

These are the **solution-specific** prerequisites for building the Shared VPC + private-access network and the VPC Service Controls perimeters. They build **on top of** the shared category requirements in `global-requirements-gcp.md` (a linked Google Cloud Organization on Cloud Identity/Workspace, Organization-level admin access, a Cloud Billing account, the `gcloud`/Python toolchain with API enablement, an agreed resource-hierarchy & naming design, and engaged security/network/governance stakeholders). Verify those first; the items below are what this solution additionally needs. Each item lists **Description / Why required / How to verify / How to implement if missing.**

1. Network administration roles at the host project and organization

• **Description:** The engineer(s) running this solution hold the network roles at the correct scope: `roles/compute.networkAdmin` and `roles/compute.securityAdmin` on the **host project** (`Net-Host – Prod – Shared-VPC`), `roles/compute.xpnAdmin` (Shared VPC Admin) at the **organization or Network folder**, and `roles/accesscontextmanager.policyAdmin` plus `roles/accesscontextmanager.policyEditor` at the organization for Access Context Manager / VPC-SC.

• **Why required:** Creating the Shared VPC and attaching service projects requires the XPN (Shared VPC) admin role; building perimeters and access levels requires the Access Context Manager policy admin role. Without them the create/attach/perimeter steps fail with permission errors.

• **How to verify:**

`gcloud organizations get-iam-policy ORG_ID --flatten="bindings[].members" --filter="bindings.members:USER"` and `gcloud projects get-iam-policy NET_HOST_PROJECT_ID …` — confirm the roles above.

• **How to implement if missing:** Have an Organization Administrator grant the roles under change control, at the **narrowest scope** that works (folder over org where possible), and record the grant in the change ticket.

2. A dedicated Shared VPC host project (and target service projects)

• **Description:** A project reserved to be the **Shared VPC host** (`Net-Host – Prod – Shared-VPC`), containing no workloads, plus the **service projects** that will attach and consume its subnets (`App-A – Prod`, `Data-A – Prod`, …). A **pilot service project** is nominated to validate on first.

• **Why required:** Shared VPC requires a clean host project to own the network; mixing workloads into the host project blurs the separation of duties this design depends on.

• **How to verify:** `gcloud compute shared-vpc get-host-project SERVICE_PROJECT_ID` (empty if not yet attached); confirm the host project exists and has the Compute Engine API enabled.

• **How to implement if missing:** Create the host and service projects under the Network/workload folders per the naming convention; enable `compute.googleapis.com` on each.

3. Non-overlapping IP address plan (subnets + secondary ranges)

• **Description:** An allocated, **non-overlapping** RFC 1918 (and/or PDP) IP plan for the regional subnets, including **secondary ranges** for GKE Pods and Services where container platforms are in scope, plan for the **restricted/private Google API VIP** routes, and reserved space for PSC.

• **Why required:** Overlapping ranges break hybrid routing (VPN/Interconnect) and VPC peering, and cannot be changed non-disruptively later. Private Google Access and PSC need known routes/ranges.

• **How to verify:** Compare the proposed CIDRs against on-prem and any existing VPCs/peered networks; confirm no overlap with `10.0.0.0/8` allocations already in use and with the Google API VIP ranges.

• **How to implement if missing:** Produce an IP allocation table (region → primary CIDR → secondary ranges) reviewed by the network team before any subnet is created.

4. Private access foundations: DNS and the restricted API VIP route

• **Description:** The ability to configure **Cloud DNS** private zones so that `*.googleapis.com`/`private.googleapis.com`/`restricted.googleapis.com` resolve to the correct VIP ranges, and to add the **route to the restricted VIP** (`199.36.153.4/30` for the restricted range — **verify the current documented VIP ranges** in Google's "Private Google Access" docs, as ranges are published there) used to keep API traffic inside a VPC-SC perimeter.

• **Why required:** Private Google Access and the VPC-SC "restricted" path only work when DNS resolves

API hostnames to the private/restricted VIP and a route exists to reach it.

• **How to verify:** From a test VM with no external IP, `nslookup storage.googleapis.com` returns the

restricted/private VIP, and a `gcloud storage ls` succeeds without public egress.

• **How to implement if missing:** Create the Cloud DNS response-policy / private zone and the static

route to the restricted VIP as part of the implementation Configure step.

5. Access Context Manager access policy (the container for access levels & perimeters)

• **Description:** An **organization-scoped access policy** in Access Context Manager (there is one

scoped policy per organization for VPC-SC use, plus optional scoped policies for delegated admin — **verify current scoping model in your org**). Perimeters and access levels live inside it.

• **Why required:** Access levels and service perimeters cannot be created without an access policy to

hold them; VPC-SC references the org access policy.

• **How to verify:** `gcloud access-context-manager policies list --organization=ORG_ID` returns a

policy (note its NAME/ID for later commands).

• **How to implement if missing:**

`gcloud access-context-manager policies create --organization=ORG_ID --title="Org Access Policy"` (Organization admin), then record the policy ID for the perimeter steps.

6. Required APIs enabled on the in-scope projects

• **Description:** The APIs this solution uses enabled on the host and in-scope projects:

`compute.googleapis.com`, `dns.googleapis.com`, `accesscontextmanager.googleapis.com`, `servicenetworking.googleapis.com` (for PSC/private services access where used), plus the APIs of the **services the perimeter will protect** (e.g. `storage.googleapis.com`, `bigquery.googleapis.com`, and `aiplatform.googleapis.com` for Vertex AI / Gemini — **verify which are VPC-SC supported for your services**).

• **Why required:** VPC-SC can only restrict a service whose API is enabled and supported; private

access and DNS steps need their APIs on.

• **How to verify:** `gcloud services list --enabled --project=PROJECT_ID` includes the APIs above.

• **How to implement if missing:** `gcloud services enable <api> --project=PROJECT_ID` for each,

under change control.

7. A logging & monitoring sink for VPC-SC and firewall audit

• **Description:** A destination (Cloud Logging bucket / log sink, and optionally BigQuery) able to receive **VPC-SC dry-run violation logs** and **firewall/VPC Flow Logs**, with query access for the engineer reviewing the pilot.

• **Why required:** The whole dry-run-first method depends on being able to **read what the perimeter would have blocked**; without the logs there is no evidence to gate enforcement on.

• **How to verify:** Confirm Cloud Audit Logs are on for the in-scope services and that `resource.type="audited_resource"` VPC-SC entries (dry-run) are queryable in Logs Explorer.

• **How to implement if missing:** Enable Data Access audit logs for the protected services, and create a log sink/bucket the reviewer can query, before turning on the dry-run perimeter.

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions

These requirements apply to the **Google Cloud Platform (GCP)** infrastructure and zero-trust solutions in the **Google** category (landing zone, Cloud IAM, networking / VPC Service Controls, Security Command Center, FinOps, and BeyondCorp Enterprise). They are the common foundation each GCP solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every GCP solution).*** *The **Google Cloud Console** (console.cloud.google.com), the `gcloud` **CLI**, and **Cloud Identity / Google Admin** are authored at exact, current click-by-click / command detail from Google's public documentation. Where a capability, API, or default is **version- or preview-dependent** (new SCC tiers, BeyondCorp features, org-policy constraints, service-perimeter behaviors), it is **cited or flagged "verify in your organization"** — never invented. **Infrastructure-as-Code** (Terraform, the Cloud Foundation Toolkit / Terraform blueprints, Config Controller) is used at the **pattern level** — the solutions give the design, the `gcloud`/console procedure, and representative IaC, and defer provider/module internals to the tool's own docs. Automation is delivered as **Python** (Cloud Client Libraries / `gcloud` wrappers), never with hardcoded credentials or keys.*

1. A Google Cloud Organization linked to Cloud Identity / Workspace

• **Description:** A **GCP Organization** resource (the root of the resource hierarchy), which exists once the domain's **Cloud Identity** or **Google Workspace** account is set up. Solutions build the folder/project hierarchy beneath it.

• **Why required:** Org-level policy, IAM, VPC Service Controls, and SCC all operate at or below the Organization node; without it there is no place to anchor centralized governance.

• **How to verify:** In the Cloud Console resource selector (or `gcloud organizations list`), confirm

an Organization resource exists and its ID.

- **How to implement if missing:** Establish Cloud Identity/Workspace for the domain so the Organization is created, then verify domain ownership.

2. Organization-level administrative access

- **Description:** An account (ideally a **break-glass super-admin** plus a delegated **Organization Administrator** / least-privilege roles the solution names) able to set org policies, create folders/projects, and grant IAM at the Organization node.
- **Why required:** Foundational changes (resource hierarchy, org policies, IAM bindings, VPC-SC perimeters) require Organization-level authority.
- **How to verify:** Confirm the account holds `roles/resourcemanager.organizationAdmin` (and the specific roles each solution lists) via `gcloud organizations get-iam-policy`.
- **How to implement if missing:** Have a super-admin grant the roles under change control; keep break-glass accounts audited and MFA-protected.

3. A Cloud Billing account

- **Description:** An active **Cloud Billing account** linked (or linkable) to the projects the solution provisions, with a billing administrator available.
- **Why required:** Projects that host resources (networks, SCC, workloads) must be linked to billing; FinOps/budget work operates directly on the billing account.
- **How to verify:** `gcloud billing accounts list` shows an OPEN account; confirm billing-admin access.
- **How to implement if missing:** Create/associate a billing account (Google or reseller) before provisioning billable resources.

4. The gcloud / IaC toolchain and API enablement

- **Description:** A workstation or pipeline with the **Google Cloud SDK (`gcloud`, `gsutil`)**, **Python 3.10+** with the **Cloud Client Libraries**, and (where used) **Terraform**; plus the ability to **enable the required Google Cloud APIs** on target projects.
- **Why required:** Configuration, automation, and the provided Python tools run through these; most services require their API to be enabled first.
- **How to verify:** `gcloud version`, `python --version`, and `gcloud services list --enabled` on a target project.

• **How to implement if missing:** Install the SDK + libraries; enable the named APIs (`gcloud services enable …`) before running the steps.

5. A resource-hierarchy, naming & environment design

• **Description:** An agreed **folder/project structure** (e.g. by department/environment), a **naming/labeling convention**, and the **environments** in scope (prod / non-prod / shared), plus a **pilot project or folder** to validate on before org-wide rollout.

• **Why required:** IAM, org policy, networking, and cost allocation all key off the hierarchy and labels; every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

• **How to verify:** Confirm the intended hierarchy, naming convention, and a named pilot scope with the platform owners.

• **How to implement if missing:** Define the hierarchy and naming standard (this is often the first solution — the Landing Zone) before building dependent controls.

6. Security, network & governance stakeholders and policy

• **Description:** Engagement with the **security (CISO/SecOps), network, and cloud-governance / FinOps** stakeholders whose policies the configuration must satisfy (least-privilege model, allowed regions, data-residency, network egress, threat-response ownership, budget/showback rules).

• **Why required:** Org policies, IAM boundaries, perimeter design, and cost governance must align to enterprise security and financial policy, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named approvers/owners for security, networking, and cost.

• **How to implement if missing:** Identify the owners and capture the policy requirements before applying org-wide guardrails.

*The deployment model across every GCP solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for org-wide changes. Automation is delivered as **Python** (`gcloud` / Cloud Client Libraries) — never with hardcoded credentials, exported service-account keys committed to disk, or secrets in code.*

Implementation

Follow these ordered steps. The house model is **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for every production-affecting change. **Every artifact is named** with the `<Purpose> – <Scope> – <Stage>` convention and the exact name is reused in testing, troubleshooting, and the scripts.

Set these shell variables once (substitute your values):

```
export ORG_ID=123456789012
export NET_HOST_PROJECT_ID=net-host-prod            # Net-Host – Prod – Shared-VPC
export PILOT_SVC_PROJECT_ID=app-a-prod              # App-A – Prod  (pilot service project)
export REGION=us-east4
export POLICY_ID=$(gcloud access-context-manager policies list \
  --organization=$ORG_ID --format='value(name)' | head -n1)
```

Artifact names used throughout:

Artifact	Name
Shared VPC host project	`Net-Host – Prod – Shared-VPC`
VPC network	`vpc-prod-shared`
Pilot subnet	`sn-prod-use4`
Hierarchical firewall policy	`HFW – Org – Baseline`
VPC network firewall policy	`NFW – Prod – Workload`
PSC endpoint (Google APIs)	`psc-googleapis`
Access level	`Access Level – Corp-Managed – v1`
Service perimeter	`Perimeter – Prod – v1`

Stage 0 — Assess (read-only)

1. Inventory the current network exposure with the assessment tool (safe, read-only):

```
python scripts/gcpnet_network_assessment.py --project $NET_HOST_PROJECT_ID --org $ORG_ID
```

It reports VPCs/subnets, whether Private Google Access is on per subnet, external IPs in use, permissive (0.0.0.0/0) firewall rules, and whether any VPC-SC perimeter already exists. Save the JSON output as the **baseline**.

1. **Gate G0 (pass/fail):** You have a written baseline of exposure and an agreed IP plan

(Requirement 3). **Fail** → do not proceed until the network team signs off the IP plan and pilot scope.

Stage 1 — Design (no changes)

1. Confirm the resource hierarchy: `Net-Host – Prod – Shared-VPC` under the **Network** folder;

`App-A – Prod` (pilot) and other service projects under the workload folders. Confirm the subnet / secondary-range table and which Google services the perimeter will protect (Requirement 6).

Stage 2 — Configure the Shared VPC + private access (pilot host first)

1. **Enable Shared VPC on the host project** (`Net-Host – Prod – Shared-VPC`):

```
gcloud compute shared-vpc enable $NET_HOST_PROJECT_ID
```

1. **Create the VPC and the pilot subnet with Private Google Access ON:**

```
gcloud compute networks create vpc-prod-shared \
  --project=$NET_HOST_PROJECT_ID --subnet-mode=custom

gcloud compute networks subnets create sn-prod-use4 \
```

```
  --project=$NET_HOST_PROJECT_ID --network=vpc-prod-shared \
  --region=$REGION --range=10.20.0.0/20 \
  --enable-private-ip-google-access --enable-flow-logs
```

Add secondary ranges (`--secondary-range pods=…,services=…`) only where GKE is in scope.

1. **Configure DNS + the restricted VIP route for private API access.** Create a Cloud DNS private

zone (or response policy) so `*.googleapis.com` resolves to the **restricted** VIP, and add the route. **Verify the current restricted/private VIP ranges in Google's "Private Google Access" documentation** before using them:

```
gcloud compute routes create rt-restricted-googleapis \
  --project=$NET_HOST_PROJECT_ID --network=vpc-prod-shared \
  --destination-range=199.36.153.4/30 --next-hop-gateway=default-internet-gateway
# Cloud DNS private zone mapping googleapis.com → restricted.googleapis.com (see docs for exact records
```

1. **(Optional, recommended) Create a Private Service Connect endpoint for the Google APIs bundle**

named `psc-googleapis`, giving an org-owned private IP for the APIs (choose the `all-apis` or `vpc-sc` bundle per your perimeter design — **verify current bundle names**):

```
gcloud compute addresses create psc-googleapis-ip \
  --project=$NET_HOST_PROJECT_ID --global --purpose=PRIVATE_SERVICE_CONNECT \
  --addresses=10.20.255.2 --network=vpc-prod-shared
gcloud compute forwarding-rules create psc-googleapis \
  --project=$NET_HOST_PROJECT_ID --global --network=vpc-prod-shared \
  --address=psc-googleapis-ip --target-google-apis-bundle=all-apis
```

1. **Provision helper (dry-run by default).** Instead of running the raw commands, you may use the

provisioning helper, which prints the plan and only mutates with `--apply`:

```
python scripts/gcpnet_shared_vpc_provision.py \
  --host-project $NET_HOST_PROJECT_ID --network vpc-prod-shared \
  --region $REGION --subnet sn-prod-use4 --range 10.20.0.0/20    # dry-run
# add --apply only after reviewing the printed plan
```

1. **Attach the pilot service project** (`App-A – Prod`) to the Shared VPC:

```
gcloud compute shared-vpc associated-projects add $PILOT_SVC_PROJECT_ID \
  --host-project=$NET_HOST_PROJECT_ID
```

Stage 3 — Firewall & egress policy (deny-by-default)

1. **Create the hierarchical firewall policy** `HFW – Org – Baseline` at the Network folder, with

deny-egress by default plus explicit allows for the Google API VIP ranges, then associate it:

```
gcloud compute firewall-policies create --organization=$ORG_ID \
  --short-name=hfw-org-baseline --description="HFW – Org – Baseline"
# add rules: allow egress to restricted VIP; deny egress 0.0.0.0/0 (low priority)
# then: gcloud compute firewall-policies associations create --folder=NETWORK_FOLDER_ID …
```

1. **Create the VPC network firewall policy** `NFW – Prod – Workload` for workload-specific allows and

associate it with `vpc-prod-shared`. Keep the default egress posture **deny**; every allow is named.

1. **Gate G1 (pass/fail):** From a pilot VM with **no external IP** on `sn-prod-use4`, private access

works (`gcloud storage ls` / API call succeeds via the restricted VIP) and unlisted egress is denied. Run `scripts/gcpnet_private_access_validate.py --project $PILOT_SVC_PROJECT_ID`.

Fail → fix DNS/route/firewall before touching perimeters. **Rollback:** the network is additive so far; detach the service project (`shared-vpc associated-projects remove`) and delete the pilot subnet/VPC if the pilot is abandoned.

Stage 4 — VPC Service Controls perimeter (DRY-RUN first)

1. **Create the access level** `Access Level – Corp-Managed – v1` describing trusted conditions (corp

IP ranges and, where BeyondCorp device posture is available, managed-device requirements — **verify device-posture availability in your org**). Author it as a `.yaml` and apply:

```
gcloud access-context-manager levels create corp_managed_v1 \
  --policy=$POLICY_ID --title="Access Level – Corp-Managed – v1" \
  --basic-level-spec=access-level-corp-managed-v1.yaml
```

1. **Create the service perimeter in DRY-RUN mode** `Perimeter – Prod – v1`, listing the in-scope

projects and the restricted services (**verify current VPC-SC coverage for each service**):

```
gcloud access-context-manager perimeters dry-run create perimeter_prod_v1 \
  --policy=$POLICY_ID --title="Perimeter – Prod – v1" \
  --perimeter-type=regular \
  --resources=projects/PILOT_PROJECT_NUMBER \
  --restricted-services=storage.googleapis.com,bigquery.googleapis.com,aiplatform.googleapis.com
```

You may instead use the builder, which emits the spec and stays in dry-run unless `--apply`:

```
python scripts/gcpnet_vpcsc_perimeter_build.py --policy $POLICY_ID \
  --title "Perimeter – Prod – v1" --resource-project-number PILOT_PROJECT_NUMBER \
  --restricted-services storage.googleapis.com bigquery.googleapis.com    # dry-run
```

1. **Let dry-run run.** For at least one full business cycle (e.g. 1–2 weeks), the perimeter **logs

what it would block **without denying anything. Review the** dry-run violation logs** (Requirement 7) with the validation report tool:

```
python scripts/gcpnet_vpcsc_perimeter_build.py --policy $POLICY_ID --show-dry-run-violations
python scripts/gcpnet_perimeter_report.py --policy $POLICY_ID --perimeter perimeter_prod_v1
```

1. **Add ingress/egress rules for legitimate flows** the dry-run surfaced (e.g. a CI/CD service

account, an on-prem source reaching BigQuery), each referencing `Access Level – Corp-Managed – v1`. Keep iterating dry-run until the only remaining violations are ones you *intend* to block.

1. **Gate G2 (pass/fail):** Dry-run violation logs contain **no legitimate flow** that would break at

enforcement (all real flows are covered by ingress/egress rules), and a **known exfiltration attempt** (e.g. copying a protected bucket to an outside project) appears as a *would-deny*. **Fail** → add the missing rule and re-run dry-run; do not enforce.

Stage 5 — Enforce (production change, with rollback)

1. **Promote the dry-run config to enforced** for `Perimeter – Prod – v1`:

```
gcloud access-context-manager perimeters dry-run enforce perimeter_prod_v1 --policy=$POLICY_ID
```

1. **Immediately re-validate:** private access still works from inside; the known exfiltration attempt

is now **actually denied** (`PERMISSION_DENIED` / `VPC Service Controls` unique ID in the error). Run `scripts/gcpnet_perimeter_report.py` again and archive the report as go-live evidence.

1. **Rollback (documented):** if enforcement breaks a legitimate flow, revert to dry-run to stop

denials immediately while you fix the rule:

```
gcloud access-context-manager perimeters dry-run drop perimeter_prod_v1 --policy=$POLICY_ID
# (returns the enforced config to dry-run behavior; add the ingress/egress rule, then re-enforce)
```

Verify the exact revert/drop subcommand in your gcloud version before you need it, and rehearse it in the pilot.

Stage 6 — Expand

1. Repeat Stages 2–5 for the remaining regions and service projects, **one wave at a time**, each

wave: attach service projects → confirm private access (G1) → add the projects to the perimeter in **dry-run** → review logs (G2) → **enforce**. Never add a project straight to an enforced perimeter — always dry-run the addition first. Bump the perimeter/access-level version suffix (`– v2`, …) when the spec materially changes, and keep the previous spec for rollback.

Testing & Validation

Run these after the corresponding implementation stage. Each test states the **procedure** and the **expected pass/fail signal**. Artifact names match the Implementation guide (`vpc-prod-shared`, `sn-prod-use4`, `Access Level – Corp-Managed – v1`, `Perimeter – Prod – v1`).

T1 — Shared VPC topology is correct

- **Procedure:**

```
gcloud compute shared-vpc get-host-project $PILOT_SVC_PROJECT_ID
gcloud compute networks subnets describe sn-prod-use4 --region=$REGION \
  --project=$NET_HOST_PROJECT_ID --format='value(privateIpGoogleAccess)'
```

- **Pass:** the host project returns `Net-Host – Prod – Shared-VPC`'s ID, and

`privateIpGoogleAccess` is `True`.

- **Fail:** empty host project (service project not attached) or `False` (private access not enabled)

→ return to Implementation Stage 2.

T2 — Private Google Access works from an internal-only VM

- **Procedure:** On a VM in `sn-prod-use4` with **no external IP**:

```
nslookup storage.googleapis.com          # expect the restricted/private VIP
gcloud storage ls                        # expect success with no public egress
python scripts/gcpnet_private_access_validate.py --project $PILOT_SVC_PROJECT_ID --region $REGION
```

- **Pass:** DNS resolves to the restricted/private VIP and the API call succeeds; the validator reports

`PRIVATE ACCESS: OK`.

- **Fail:** DNS resolves to a public IP, or the call times out / needs an external IP → fix the Cloud

DNS zone and the restricted-VIP route (Implementation step 6).

T3 — Egress is deny-by-default

- **Procedure:** From the same internal-only VM, attempt egress to an arbitrary public host not on any allow rule (e.g. `curl -m 5 https://example.com`).
- **Pass:** the connection is **blocked/timed out** (no matching allow in `HFW – Org – Baseline` / `NFW – Prod – Workload`), while Google API access (T2) still works.
- **Fail:** arbitrary egress succeeds → the deny-egress baseline is missing or out-prioritized; review the firewall policies.

T4 — Perimeter dry-run logs a known exfiltration attempt

- **Procedure:** With `Perimeter – Prod – v1` in **dry-run**, attempt to copy a protected bucket to a project **outside** the perimeter:

```
gcloud storage cp gs://protected-bucket/obj gs://outside-bucket/obj    # from inside perimeter
python scripts/gcpnet_vpcsc_perimeter_build.py --policy $POLICY_ID --show-dry-run-violations
```

- **Pass:** the copy **succeeds** (dry-run does not deny) but a **would-deny** VPC-SC violation for that request appears in the dry-run logs / tool output.
- **Fail:** no violation logged → the perimeter does not cover `storage.googleapis.com`, the project is not in `--resources`, or audit logging is off (Requirement 7).

T5 — Legitimate flows are covered before enforcement (Gate G2)

- **Procedure:**

```
python scripts/gcpnet_perimeter_report.py --policy $POLICY_ID --perimeter perimeter_prod_v1
```

Review the report's classification of dry-run violations into **legitimate (needs a rule)** vs **intended-block**.

- **Pass:** every legitimate flow has a matching ingress/egress rule referencing `Access Level – Corp-Managed – v1`; only intended exfiltration-style violations remain.
- **Fail:** any legitimate business flow still shows as would-deny → add the ingress/egress rule and re-run dry-run. **Do not enforce.**

T6 — Enforcement blocks exfiltration and preserves legitimate access

- **Procedure:** After `perimeters dry-run enforce`, repeat T4's exfiltration copy and a legitimate in-perimeter API call.
- **Pass:** the exfiltration copy now **fails** with a `VPC Service Controls` `PERMISSION_DENIED` (note the unique violation ID), while the legitimate in-perimeter call still **succeeds**.
- **Fail:** exfiltration still succeeds (not enforced / wrong service) **or** a legitimate call now fails (missing ingress/egress rule) → for the latter, execute the **rollback to dry-run** (Implementation step 20), add the rule, and re-enforce.

T7 — Access level gates by trusted condition

- **Procedure:** Trigger a request that satisfies `Access Level – Corp-Managed – v1` (from a corp IP /

managed device) and one that does not, against a service governed by an ingress rule.

- **Pass:** the satisfying request is allowed; the non-satisfying request is denied by VPC-SC.
- **Fail:** both allowed → the ingress rule is not actually referencing the access level; both denied →

the access level's conditions are too strict or misconfigured (verify device-posture availability in your org).

T8 — Config helper output matches live state (drift check)

- **Procedure:**

```
python scripts/gcpnet_config_helper.py --org $ORG_ID --host-project $NET_HOST_PROJECT_ID \
  --policy $POLICY_ID --check
```

- **Pass:** the helper reports the expected artifacts present and named

(`vpc-prod-shared`, `sn-prod-use4`, `Perimeter – Prod – v1`, `Access Level – Corp-Managed – v1`) with no drift from the intended config.

- **Fail:** the helper flags a missing/renamed artifact or drift → reconcile against the Implementation

guide before expanding to the next wave.

Troubleshooting

Common failure modes and how to resolve them. Artifact names match the Implementation guide (`vpc-prod-shared`, `sn-prod-use4`, `Access Level – Corp-Managed – v1`, `Perimeter – Prod – v1`).

Private access

Symptom: DNS resolves `storage.googleapis.com` to a *public* IP, or API calls from an internal-only VM time out.

- **Cause:** Cloud DNS is not mapping `googleapis.com` to the restricted/private VIP, the restricted-VIP

route is missing, or Private Google Access is off for the subnet.

- **Resolve:** Confirm `privateIpGoogleAccess=True` on `sn-prod-use4` (test T1); verify the Cloud DNS

private zone / response policy points `*.googleapis.com` at the **restricted** VIP; confirm the route to the restricted VIP range exists. **Verify the current restricted VIP range in Google's "Private Google Access" documentation** — do not assume a hard-coded value. Re-run `gcpnet_private_access_validate.py`.

Symptom: API access works but only when the VM has an external IP.

- **Cause:** Traffic is going out the public path, not the private VIP — private access is not actually

in effect.

• **Resolve:** Remove the external IP and re-test; fix DNS/route as above until the call succeeds with internal IP only.

Shared VPC

Symptom: `shared-vpc associated-projects add` **fails, or the service project cannot use subnets.**

• **Cause:** Missing `roles/compute.xpnAdmin` (Shared VPC Admin) at the org/folder, Shared VPC not enabled on the host, or the service account lacks subnet IAM.

• **Resolve:** Confirm Shared VPC is enabled on `Net-Host – Prod – Shared-VPC` (`gcloud compute shared-vpc get-host-project …`); grant `compute.xpnAdmin` at the narrowest scope (Requirement 1); grant subnet-level `networkUser` on the shared subnet to the service project's service accounts.

Symptom: hybrid/VPN/peering routes fail or behave oddly after adding a subnet.

• **Cause:** Overlapping IP ranges with on-prem or a peered VPC (Requirement 3).

• **Resolve:** Audit the IP plan for overlap; ranges cannot be renumbered non-disruptively — plan a migration for the overlapping subnet rather than patching routes.

Firewall / egress

Symptom: arbitrary public egress still succeeds from a workload.

• **Cause:** The deny-egress baseline in `HFW – Org – Baseline` is missing, out-prioritized by a broader allow, or the policy is not associated with the folder/VPC.

• **Resolve:** Confirm the hierarchical policy is associated and that the deny-egress rule is at a lower (broader) priority number than any 0.0.0.0/0 allow; ensure the only egress allows are the named ones (Google API VIP ranges, explicit destinations).

VPC Service Controls — dry-run

Symptom: the known exfiltration test (copy to an outside project) produces *no* dry-run violation.

• **Cause:** The service (`storage.googleapis.com`) is not in the perimeter's restricted list, the source/target project is not in `--resources`, or Data Access **audit logging is off** for the service (Requirement 7).

• **Resolve:** Add the service to `restricted-services`; add the project number to the perimeter's resources (in dry-run); enable Data Access audit logs for the protected services, then regenerate the test request and re-check with `gcpnet_vpcsc_perimeter_build.py --show-dry-run-violations`.

Symptom: dry-run shows would-deny violations for flows you believe are legitimate.

• **Cause:** A real business flow (CI/CD service account, on-prem source, cross-project pipeline) is not

yet covered by an ingress/egress rule.

- **Resolve:** Add a **narrow** ingress/egress rule referencing `Access Level – Corp-Managed – v1`, the

specific identity, and the specific service — do not widen the perimeter or drop a restricted service. Re-run dry-run until only intended blocks remain (Gate G2).

VPC Service Controls — enforced

Symptom: after enforcing, a legitimate call fails with `VPC Service Controls` / `PERMISSION_DENIED` and a unique violation ID.

- **Cause:** A legitimate flow was not covered before enforcement (Gate G2 was passed on incomplete

logs).

- **Resolve:** Execute the documented **rollback to dry-run** immediately to stop denials

(`perimeters dry-run drop …` — **verify the exact subcommand for your gcloud version**), add the missing ingress/egress rule, confirm in dry-run, then re-enforce. Use the unique violation ID in the error to pinpoint the blocked request in the logs.

Symptom: enforcement did not seem to take effect (exfiltration still succeeds).

- **Cause:** The `dry-run enforce` step was not run, or a spec/status mismatch left the perimeter still

in dry-run.

- **Resolve:** Run `gcpnet_perimeter_report.py` — if it reports `DRY-RUN`, the enforce step did not

promote the spec; re-run `perimeters dry-run enforce perimeter_prod_v1`.

Permissions & policy

Symptom: `access-context-manager` commands fail with permission denied.

- **Cause:** Missing `roles/accesscontextmanager.policyAdmin` / `policyEditor`, or the wrong/absent

org access policy ID.

- **Resolve:** Confirm the org access policy exists

(`gcloud access-context-manager policies list --organization=ORG_ID`) and that the running identity holds the policy roles (Requirement 1 & 5); pass the correct `--policy` ID.

Tooling

Symptom: a Python tool prints `gcloud not found on PATH` or empty results.

- **Cause:** Google Cloud SDK not installed / not on PATH, or ADC not configured.
- **Resolve:** Install the SDK and run `gcloud auth application-default login` (or use the pipeline's

Workload Identity); confirm `gcloud version`. The tools are read-only/dry-run by default, so re-running after fixing auth is safe.

Best Practices

Durable practices for operating and evolving this network and its VPC Service Controls perimeters.

Perimeters

- **Always change perimeters in dry-run first.** Every material change — adding a project, adding a restricted service, tightening an access level — goes through `dry-run` and a log review before it becomes enforced. Never add a project directly to an enforced perimeter.
- **Version your specs.** Keep the previous perimeter/access-level spec (`– v1`, `– v2`) so rollback is a known-good artifact, not an improvisation. Store the go-live report (from `gcpnet_perimeter_report.py`) with the change record.
- **Keep a rehearsed rollback.** The revert-to-dry-run step is the emergency brake; rehearse it in the pilot and confirm the exact `gcloud` subcommand for your version *before* you need it.
- **Prefer ingress/egress rules over widening the perimeter.** When a legitimate flow is blocked, add a narrow ingress/egress rule referencing a specific access level, service, and identity — do not drop a service from the restricted list or add a broad project just to make an error go away.
- **Re-verify service coverage periodically.** VPC-SC support for a service (supported / preview / unsupported) changes; recheck coverage for your services — especially Vertex AI / Gemini — on a schedule, and adjust the restricted-services list accordingly.

Access levels & identity

- **Model trust with device + network posture, not IP alone.** Where BeyondCorp device posture is available in your org, combine corp IP ranges with managed-device requirements in `Access Level – Corp-Managed – v1`; IP-only access levels are weaker.
- **Scope access levels tightly and name them by intent.** One access level per trust profile, reused by reference across ingress rules — avoid per-rule copies that drift.

Shared VPC & private access

- **Host project holds no workloads.** Keep the separation of duties clean: network in the host project, apps in service projects.
- **Enable Private Google Access on every subnet and remove external IPs by default.** Reach Google APIs via the restricted VIP / PSC; grant external IPs only by exception with justification.
- **Deny egress by default.** Build up from a deny-egress hierarchical policy; every allow is named, justified, and reviewed. Turn on VPC Flow Logs and firewall logging for evidence.

• **Plan IP space once, generously.** Non-overlapping ranges with room to grow and reserved secondary

ranges — you cannot renumber non-disruptively later.

Automation & secrets

• **Run tools read-only / dry-run first; require an explicit** `--apply`**.** The included tools follow this

pattern — keep it for any extension you write.

• **Never commit credentials or service-account keys.** Use Application Default Credentials / the

environment / Workload Identity; keep keys out of code and out of the repo.

• **Use change control for org-wide guardrails.** Hierarchical firewall policies and perimeter

enforcement affect many projects — land them under ticketed change with the named security/network approvers.

Operations

• **Route VPC-SC and firewall logs to a queryable sink** and alert on unexpected *enforced* denials —

they indicate either an attack or a missing legitimate-flow rule.

• **Expand in waves with the same gates.** Each region/service-project wave repeats

attach → private-access check (G1) → dry-run perimeter add → log review (G2) → enforce. Consistency is what keeps a large rollout safe.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

Document	What it gives you
Solution Overview	The network reference design: Shared VPC topology, private access (Private Google Access / PSC), hybrid connectivity, firewall/egress model, and VPC Service Controls perimeter design — with benefits, limitations, and the deployment model.
Requirements Guide	Solution-specific prerequisites (network admin roles, host/service projects, IP plan, private-access DNS/route, Access Context Manager policy, required APIs, audit-log sink) — each with how to verify and how to implement if missing — on top of the shared GCP category requirements.

Document	What it gives you
Implementation Guide	Exact, ordered, named steps: build the Shared VPC + private-access subnet, apply deny-by-default firewall/egress policy, and design + **dry-run then enforce** VPC-SC perimeters — pilot-first, with pass/fail gates (G0–G2) and a documented rollback.
Testing & Validation	Eight tests (T1–T8) with expected pass/fail signals, from Shared VPC correctness and private access through the dry-run exfiltration test to enforced-perimeter validation and drift checking.
Troubleshooting	Common failure modes (private access resolving public, perimeter not logging, enforcement breaking a flow, permission errors) and their resolutions.
Best Practices	Durable practices for evolving perimeters safely, access-level design, private access, and safe automation.
Executive Summary	The outcome, approach, and why it is safe/governed — for decision-makers.

Python tools (5) — `gcloud` / Cloud Client Library automation

All are **read-only or dry-run by default**, require an explicit `--apply` for any write, use Application Default Credentials, and contain **no hardcoded credentials or keys**. Python 3.10+.

#	Tool	What it does
1	`gcpnet_network_assessment.py`	Read-only exposure baseline: VPCs/subnets, subnets missing Private Google Access, 0.0.0.0/0 firewall rules, external IPs, and whether any VPC-SC perimeter exists. Emits JSON.
2	`gcpnet_shared_vpc_provision.py`	Provisioning helper (dry-run default): enables Shared VPC on the host project and creates the VPC + private-access subnet; prints the exact plan and only mutates with `--apply`.
3	`gcpnet_private_access_validate.py`	Read-only validator that Private Google Access is on per subnet, the restricted-VIP route exists, and (from an internal-only VM) API hostnames resolve to a private VIP.

#	Tool	What it does
4	`gcpnet_vpcsc_perimeter_build.py`	Builds a VPC-SC service perimeter **in dry-run mode first** (creates only with `--apply`, still dry-run; never enforces), and reads back dry-run would-deny violation logs.
5	`gcpnet_perimeter_report.py`	Read-only go/no-go report: perimeter mode (dry-run vs enforced), restricted services, resources, access levels, and a dry-run violation count — the evidence for Gate G2.

(Bonus config/scope helper: `gcpnet_config_helper.py` emits the intended artifact config and runs a read-only drift check against live state — used by test T8.)

Named artifacts (reused across docs, tests, and tools)

`Net-Host – Prod – Shared-VPC` · `vpc-prod-shared` · `sn-prod-use4` · `HFW – Org – Baseline` · `NFW – Prod – Workload` · `psc-googleapis` · `Access Level – Corp-Managed – v1` · `Perimeter – Prod – v1`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

gcpnet_config_helper.py

gcpnet_network_assessment.py

gcpnet_perimeter_report.py

gcpnet_private_access_validate.py

gcpnet_shared_vpc_provision.py

gcpnet_vpcsc_perimeter_build.py